

1. Low-Rank Adaptation (LoRA)

Full fine-tuning of large pre-trained models carries a high risk of catastrophic forgetting on small datasets. In order to address this, the researchers utilized a technique called Parameter Efficient Fine Tuning (PEFT), specifically, Low-Rank Adaptation (LoRA) (Hu et al., 2022) which freezes the original model weights and injects trainable low-rank matrices into each attention layer:

$$W' = W + \Delta W = W + BA$$

Equation 1. LoRA Weight Update

Where W represents the frozen pre-trained weight matrix, while B and A are the newly introduced trainable decomposition matrices. These are smaller matrices that act as a bottleneck.

Hypothetically, if an attention layer has a dimension of $10,000 \times 10,000$, LoRA reduces computational load by restricting the parameter updates to a much smaller subspace Rank (r). If $r = 32$, the decomposition operates as follows:

Matrix A: Projects the input dimension down to the lower-rank space ($10,000 \times 32$), resulting in 320,000 parameters.

Matrix B: Conversely, projects the lower-rank representation back up to the original dimensionality ($32 \times 10,000$), resulting in an additional 320,000 parameters.

This gives us a total of 640,000 trainable parameters instead of updating millions of parameters across the full architecture.

The LoRA update is scaled by a factor α/r before being added to the frozen weights:

$$W' = W + (\alpha/r) \cdot BA$$

Equation 2. Scaled LoRA Update.

While Rank (r) controls the dimensions of the subspace, Alpha (α) controls the magnitude of the adaptation. A higher α gives the LoRA update more influence over the final output, while a lower α conserves the original weights.

When applied to the study's experimental configuration, the efficiency of this approach was demonstrated clearly. The FG-CLIP base architecture has a total of 149,620,737 parameters. Through LoRA, the configurations tested required updating only roughly 0.49 million to 3.93 million

parameters which represents only 0.33% to 2.63% of the entire architecture. This prevents the neural network from severely overfitting to the small dataset while preserving the model's pretrained knowledge.

2. Dataset and Data Preparation

2.1 Dataset Construction

Training consisted of 420 image pairs (840 images total) gathered from e-commerce platforms such as Facebook Marketplace, Shopee, and Carousell. This approach helped capture the variable lighting, backgrounds, and camera angles typical of real-world lost and found images. Each pair consists of one image labeled as the "lost" item, and another image of the same item labeled as the "found" item. The dataset included 6 categories: Bags, Chargers, Handkerchiefs, Lunchboxes, Tumblers, and Wallets.

The dataset was partitioned as follows
:

Table 2.1-1. Data Split per Category (Train–Validation–Test)

Category	Total Pairs	Train (70%)	Validation (15%)	Test (15%)
Bags	100	70	15	15
Chargers	100	70	15	15
Handkerchiefs	100	70	15	15
Lunchboxes	100	70	15	15
Tumblers	100	70	15	15
Wallets	100	70	15	15
	600	420	90	90

The table above shows how the 600 image pairs are divided across the six item categories. Each category contains exactly 100 pairs, which are then split 70/15/15 into training, validation, and test sets. The balanced distribution ensures that all categories are equally represented during training and evaluation.

2.2 Caption Generation and Textual Hard Negatives

For image captioning, the researchers utilized a vision-language model (VLM), Claude AI to generate descriptions for all 600 pairs which were then verified by a human annotator. Each caption follows a structured attribute taxonomy:

[Color] + [Brand/Text] + [Specific Category] + [Condition/Distinguishing Feature]

For example: *"Pink Guess compact zip-around wallet, quilted diamond-stitched faux leather, silver triangle GUESS logo plate centered on front flap."*

For each positive caption (an accurate description of the item), a corresponding negative caption was constructed by modifying one defining attribute/feature such as color, material, hardware, texture or shape, while keeping the rest of the caption identical. This serves as our Textual Hard Negative, forcing the model's text encoder to differentiate at a microscopic level rather than relying on broad semantic categories.

Intersection Rule

An intersection rule was introduced to control which attributes appear in the positive captions. The rule dictates that only features visible in both images of a pair are described. This prevents the text encoder from referencing attributes the vision encoder cannot observe, preventing model hallucination and ensuring that both encoders are grounded in mutually visible evidence.

2.3 Data Augmentation

Training images pass through augmentation pipeline to simulate the variability of typical user-submitted lost and found photographs:

- Resize to 224×224
- Random horizontal flip: simulates different viewing angles and orientations
- Color jitter: changes brightness, contrast, saturation, and hue to account for camera and lighting variation
- Random erasing: randomly masks a rectangular region, preventing over-reliance on a single salient feature (e.g., a brand logo) and encouraging the model to leverage shape, texture, and material cues holistically

Validation and test images receive only the resize operation to ensure unbiased performance evaluation.

3. Domain Specific Training Methodology

3.1 Fine-Tuning Pipeline: Processing Stages

The FG-CLIP fine-tuning process involves a four-stages: (1) Batch Sampling, (2) Dual-Stream Encoding, (3) Similarity Computation and Contrastive Loss, (4) Backpropagation and Weight Update.

Stage 1: Batch Sampling and Construction

Before the model begins performing any computations, the training data is organized into batches. For this stage, two main strategies were considered: Random Shuffle (default) and Categorical Batch Sampling. The default approach shuffles all 420 training pairs and divides them into fixed-sized batches in random order. This is the standard PyTorch DataLoader behavior.

Custom Batch Sampling provides a more targeted approach, actively constructing batches to maximize visual difficulty. The custom PyTorch Sampler subclass operates in three phases per epoch:

Phase 1: Grouping

All 420 training pairs are partitioned by their category, extracted from the image filename (e.g “tumbler_47” → “tumbler”). This produces six lists, one per category (Bags, Chargers, Tumblers, Wallets, Handkerchiefs, Lunchboxes) each containing 70 indices.

```
# filename format: "lost_{category}_{number:02d}.png"
for idx, row in dataset_df.iterrows():
    filename = row['lost_image'] # e.g. "lost_lunchbox_01.png"
    category = filename.split('_')[1] # e.g. "lunchbox"
    category_to_indices.setdefault(category, []).append(idx)

# Result:
# {"bag": [14, 37, 55, 81, ...], # 70 indices
# "charger": [3, 18, 29, 46, ...],
# "handkerchief": [7, 24, 41, 63, ...],
# "lunchbox": [1, 9, 16, 27, ...],
# "tumbler": [2, 8, 15, 26, ...],
# "wallet": [6, 11, 20, 32, ...]}
```

Pseudocode 3.1-1. Batch Grouping Internal Logic

The figure above shows the internal logic behind phase 1. The code iterates over every row in the training split of the dataset and extracts the lost_image filename, splits it on underscores, and takes the second token as the category name. The resulting dictionary maps each category to a list of all dataset indices belonging to it. The setdefault pattern appends each index to the list for that category.

Phase 2: Chunking

Within each category list, indices are randomly shuffled. Each shuffled list is then divided into fixed-sized chunks that is equal to the batch size. The final chunk is discarded if it has only 1 item.

```
batch_size = 16
batches = []
for cat, indices in category_to_indices.items():
    random.shuffle(indices)
    chunks = [indices[i:i + batch_size]
              for i in range(0, len(indices), batch_size)]
    for chunk in chunks:
        if len(chunk) > 1: # discard single-item chunks
            batches.append(chunk)

# Example - "lunchbox" shuffled: [27, 9, 1, 16, ...]
# Chunked at batch_size=16: [[27, 9, 1, 16, ..., 3], [22, 38, ...], ...]
# Only chunks with len > 1 survive → added to batches
```

Pseudocode 3.1-2. List Chunking Internal Logic

The figure below shows the internal logic behind stage 2. The chunking loop takes each category's shuffled index list and steps through it in increments of `batch_size`, producing sublists of that length. The line `if len(chunk) > 1` drops any remainder smaller than a full batch.

Phase 3: Interleaving

All chunks from all categories are collected into a single flat list of batches then shuffled a second time. This ensures consecutive batches do not always come from the same category.

```
random.shuffle(batches)

# Before shuffle: [batch_from_lunchbox, batch_from_lunchbox, batch_from_bag, ...]
# After shuffle: [batch_from_bag, batch_from_tumbler, batch_from_lunchbox, ...]
```

Pseudocode 3.1-3. Batch Interleaving Internal Logic

The figure above shows internal logic behind phase 3. After chunking, a second shuffle over the full batch list mixes them so that no two consecutive batches in an epoch are from the same category, preventing the model from learning patterns that depend on batch order.

Grouping same-category items into each batch makes every hard negative a greater possible match for every image in the batch. This creates a more difficult discrimination task than uniform random shuffling, forcing the vision encoder to learn micro-features rather than relying on macro features (general shape, color etc)

Stage 2: Dual-Stream Encoding

ViT-B/16 vision encoder + transformer text encoder

Each batch contains N images, N positive captions, and N hard-negative captions (2N total text descriptions). Both streams are encoded simultaneously via the LoRA-adapted FG-CLIP model's dual-stream encoder architecture (ViT-B/16 vision encoder + transformer text encoder) which was described in detail in Section 3 of the main paper.

LoRA integration

LoRA adapters are injected into the attention projection matrices q_proj , k_proj , v_proj , and out_proj of every transformer layer in both the vision and text encoders. For each frozen weight matrix $W \in \mathbb{R}^{d \times k}$, two trainable matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ are introduced. During the forward pass, the output is:

$$W'(x) = Wx + \frac{\alpha}{r} \cdot BAx$$

Equation 3. Lora Forward Pass

where W is the frozen pre-trained weight matrix, B and A are the two trainable low-rank matrices, α controls how strongly the LoRA update influences the output, and r is the rank of the low-rank subspace.

Output of Stage 2. Two embedding tensors: N normalized image embeddings and 2N normalized text embeddings, each of dimension 512, in the shared FG-CLIP embedding space.

Stage 3: Similarity Computation and Contrastive Loss

The embeddings from Stage 2 are already normalized, but the model additionally applies explicit L2-normalization before computing the image-text similarity matrix:

```
image_embeds = outputs.image_embeds / ||outputs.image_embeds||_2
text_embeds = outputs.text_embeds / ||outputs.text_embeds||_2
```

Pseudocode 3.3-1. L2 Normalization

The code above illustrates how both image and text embeddings are L2-normalized, they are scaled to have a unit length of 1 so that their dot product directly results in a cosine similarity score between -1 and 1 . This prevents differences in embedding magnitude from affecting similarity scores, only the direction of each embedding vector is considered.

Cosine similarity matrix (Training)

The $N \times 2N$ similarity matrix S is computed as the dot product of image and text embeddings:

$$S_{ij} = \frac{(e_{img} \cdot e_{text})^T}{T}$$

Equation 4. Cosine Similarity Matrix During Training

Where T is the temperature, i is the row index (image index, $i = 1 \dots N$), and j is the column index (caption index, $j = 1 \dots N$). The first N entries correspond to the positive captions paired with each image, and the last N entries correspond to the hard negative captions for each image. The temperature T controls how spread out the similarity scores are. A smaller T makes the scores more spread apart, highlighting the differences between similar and dissimilar pairs more sharply, while a larger T compresses all scores closer together, making them more uniform.

The equation above defines how each entry in the $N \times 2N$ similarity matrix is computed. Each row represents one image, and each column represents one caption. The dot product measures how aligned the image and text embeddings are, then the result is divided by temperature T to control the sharpness of the similarity distribution. Note that normalization denominator is omitted here because both embeddings are already L2-normalized in Stage 3 ($\|i_i\| = \|t_j\| = 1$). The general cosine similarity form with the denominator is given in Equation 3.5 of the main paper document.

After encoding, and normalization, each image has a 512-dimensional embedding e_{img} and each caption has a 512-dimensional embedding e_{text} . The similarity between a given image and caption is computed as the dot product of their normalized embeddings, scaled by temperature τ :

For a positive pair (the correct caption for the image), the similarity might be:

$$S_{correct} = (0.42)(0.45) + (0.38)(0.41) + (0.29)(0.32) + \dots (512 \text{ dimensions}) = 0.8231$$

After dividing by temperature $\tau = 0.03$, the logit becomes:

$$logit = 0.8231 / 0.03 = 27.44$$

For a hard negative (a caption that differs by one attribute, e.g., dark grey instead of light grey), the same computation results in a lower alignment:

$$S_{negative} = (0.30)(0.45) + (0.25)(0.41) + (0.20)(0.32) + \dots = 0.6519$$

$$logit = 0.6519 / 0.03 = 21.73$$

The positive pair produces a higher logit meaning the model assigns a higher probability to the correct match. The temperature scaling amplifies this difference. A lower temperature like 0.03 produces sharper, more spread-out logits, controlling how strongly the model is penalized for confusing a positive with a hard negative.

Cross Entropy Loss

The standard CLIP contrastive loss is applied as a symmetric cross entropy over the similarity matrix:

```
labels = torch.arange(N).to(device) # diagonal = correct matches
loss = CrossEntropyLoss(logits, labels) # logits = S, shape (N, 2N)
```

Pseudocode 3.3-2. Contrastive Lost Computation

The code above shows how the contrastive loss is computed in PyTorch. The labels tensor creates a sequence from 0 to N-1, representing the index of the correct (positive) caption for each image. PyTorch's CrossEntropyLoss then treats each row as an independent classification problem, where the correct answer is always at the diagonal position. Image 0 should match caption 0, then image 1 should match caption 1 etc. Meanwhile, the hard negative captions occupy the off-diagonal positions.

Stage 4: Backpropagation and Weight Update

Gradient flow

The loss value is passed backwards through the entire model from the similarity scores, through the text encoder, and through the vision encoder. This backward pass computes how much each trainable parameter contributed to the error. Only the LoRA adapter matrices A and B receive these gradient updates, while the original pre-trained model weights remain intact throughout the fine-tuning process.

Optimizer: AdamW

AdamW is the optimization algorithm used for adjusting the LoRA matrices after each backward pass. It is applied only to the trainable parameters (LoRA weights). Its key settings are:

- Learning rate (LR): Controls how large each weight update step is.
- Weight decay: A small penalty that discourages the LoRA weights from growing to large. This is typically used to keep the model stable and help it generalize better.
- Betas: Internal tracking variables that give the optimizer memory. Smooths out gradient directions, and betas[1] smooths out gradient magnitudes, so the optimizer can make more informed updates over time.
- Epsilon: A tiny constant added to the denominator during gradient updates to prevent division by zero

Learning rate schedule

Instead of using a fixed learning rate throughout training, the learning rate changes over time in two phases:

Phase 1: Warmup (first 10% of steps)

The learning rate starts at 0 and gradually increases to its peak value. At the start of training, the LoRA matrices are near zero, so applying a learning rate immediately could potentially cause unstable updates.

Phase 2: Cosine decay (first 10% of steps)

The learning rate smoothly decreases from its peak value down to roughly half of the peak. This lets the model make large, confident updates early on to learn quickly, then slowly refine its weights as it gets closer to the best solution

Early stopping

After every training epoch, the model is evaluated on the validation set (with gradients disabled). If the validation loss does not improve for a set number of epochs, the training stops automatically. The LoRA adapter weights that produced the lowest validation loss across all epochs are the ones that are saved using Hugging Face's PEFT library. The library exports these as two files: adapter_model.safetensors (the actual weight values) and adapter_config.json (the configuration).

Table 3.1-1. Stage Progression Summary

Stage	Input	Operation	Output
1	420 dataset pairs	Random shuffle (default) or Categorical Batch Sampler (hard negative mining)	N image-caption pairs per step
2	N images + 2N captions	LoRA-adapted dual-stream encoding	N image embeds (512-d) + 2N text embeds (512-d)

		(ViT-B/16 + text transformer)	
3	All embeddings	L2-normalize $\rightarrow$ cosine sim $\rightarrow T \rightarrow$ CE loss over $(N \times 2N)$ matrix	Scalar loss
4	Loss scalar	AdamW + cosine LR + 10% warmup + early stopping	LoRA weight updates ($\Delta W = BA$)

The table above illustrates the complete four-stage fine-tuning pipeline. Each stage takes a specific input, performs a defined operation, and produces a defined output that feeds into the next stage. Stages 1 through 4 represent the data flow from raw dataset pairs to updated LoRA weights.

3.2 Contrastive Loss

For a batch of N image-caption pairs, the model receives N images alongside $2N$ text descriptions (N positives + N hard negatives). All embeddings are L2-normalized before computing similarity logits. The contrastive loss penalizes the model when a hard-negative description receives a higher similarity score to an image than its correct positive caption. This training setup is consistent with the hard-negative component of FG-CLIP's pre-training but applied to the campus lost and found domain.

3.3 Categorical Batch Mining (Visual Hard Negatives)

While the dataset provides textual hard negatives, standard randomized data loading fails to consistently challenge the vision encoder, as visually dissimilar items (e.g., a bag and a charger) are easily distinguished. To address this, we implemented a custom Categorical Batch Sampler for our advanced training configurations.

Instead of randomly shuffling the entire dataset, the sampler actively "mines" the dataset to construct batches consisting entirely of items from the same category (e.g., a batch of 16 highly similar lunchboxes). By forcing visually similar items into the same batch, the model is penalized for relying on macro-features like general shape or dominant color. This forces the vision encoder to visually map the micro-features detailed in the textual hard negatives (such as a specific latch or zipper) to lower the contrastive loss.

4 Results and Discussion

4.1 FG-CLIP

4.1.1 Experimental Configurations

Three fine-tuning configurations were developed to trace the impact of each design decision:

Table 4.1.1-1. Three experimental fine-tuning configurations of LoRA

Hyperparameter	V1	V2	V2.1
LoRA rank (r)	8	32	32
LoRA alpha (α)	16	64	128
Target modules	qproj, vproj	qproj, kproj, vproj, outproj	qproj, kproj, vproj, outproj
LoRA dropout	0.1	0.1	0.1
Trainable parameters	491,520 (0.33%)	3,932,160 (2.56%)	3,932,160 (2.56%)
Optimizer			
Optimizer	AdamW	AdamW	AdamW
Learning rate	5e-5 (fixed)	5e-5	3e-5
Scheduler	None	Cosine + 10% warmup	Cosine + 10% warmup
Weight decay	0.01	0.01	0.01
Loss	Single-image	Single-image	Single-image
Temperature (τ)	~0.07 (model default)	0.02 (fixed)	0.03 (fixed)
Loss type	Single-image	Single-image	Single-image
Batch size	16	16	8
Batching strategy	Random shuffle	Category-grouped	Category-grouped
Early stopping	None	Yes	None
Max epochs	15	20	20

Hyperparameter	V1	V2	V2.1
Color jitter	(brightness=(0.8, 1.2), contrast=None, saturation=None, hue=None)	(brightness=0.2, contrast=0.2, saturation=0.4, hue=0.1)	(<i>brightness=0.4,</i> <i>contrast=0.4,</i> <i>saturation=0.4,</i> <i>hue=0.1)</i>

The table above compares the three experimental configurations side by side across all key dimensions: LoRA architecture settings (rank, alpha, target modules), optimizer settings (learning rate, scheduler, weight decay), loss settings (temperature, loss type), and training settings (batch size, batching strategy, early stopping, max epochs, and data augmentation). V1 establishes a conservative baseline. V2 demonstrates an expanded LoRA capacity with category-grouped batching. V3 applies a smaller LoRA rank ($r=16$) with stronger weight decay (0.1) and a higher temperature (0.05).

4.1.2 Key Design Decisions

V1 → V2: Architecture and Training

V1 established a baseline using conservative LoRA settings (rank 8, q/v projections only, no scheduler, brightness-only augmentation). V2 introduced a suite of changes to the architecture, training procedure, and data pipeline:

- **LoRA rank 8 → 32, q/v → q/k/v/out.** Expanding both the rank and target modules gives the adapter significantly more capacity to learn fine-grained visual-textual alignment. Adding kproj and outproj enables the adapter to directly modify the query-key computation which is how the model decides which tokens to attend to rather than only adjusting the value aggregation that follows. This increases trainable parameters from ~0.33% to ~2.56% of the full model.
- **Random shuffling → Category-grouped batching.** The custom Categorical Batch Sampler groups pairs by category prefix (e.g. bag_012 → "bag"). Each category's shuffled index list is chunked into batches of 16. Every training batch therefore contains 16 pairs from the same category, with all 16 hard negatives being plausible matches for each image, creating a much harder discrimination task than random batching
- **Temperature 0.07 → 0.02.** A lower temperature sharpens the softmax distribution, resulting in steeper gradient signals and a stricter penalty for placing probability mass on hard negatives.

- **Fixed LR** → **Cosine scheduler with warmup**. A 10% linear warmup period stabilizes early training when the LoRA matrices are near-zero initialization. The cosine decay enables smooth convergence without abrupt learning rate drops.
- **Brightness-only** → **Full-spectrum ColorJitter**. V1 only jittered brightness, leaving contrast and saturation invariant. V2 + HNM applies brightness, contrast, saturation, and hue jitter, giving a more uniform robustness envelope against real-world lighting variation.
- The combined effect was a substantial improvement from V1: +7.78 on R@1 (71.11% → 78.89%), +3.34 on R@10 (93.33% → 96.67%), and -3 severe failures (6 → 3).

V2 → V2.1: Optimizing Training

While V2 outperformed V1 through expanded LoRA capacity and category-grouped batching, the combination of $r=32$ with $wd=0.01$ introduced a moderate overfitting tendency: training loss reached 0.2818 while validation loss remained at 1.1819 at epoch 10, compared to best checkpoint at epoch 7 (train loss 0.4608, val loss 1.1684). V3 applies targeted regularization to unlock the best generalization:

- **$r = 32$ → 16**. Halving the LoRA rank acts as a structural regularizer, reducing the adapter's capacity to memorize training pairs. This forces the model to learn more generalizable visual-text alignments rather than overfitting to specific examples.
- **$\alpha = 64$ → 32**. Keeping α/r constant at 2 maintains the same effective scaling behavior while reducing the absolute magnitude of LoRA updates.
- **Weight decay 0.01 → 0.1**. Ten times stronger weight decay penalizes large LoRA weights, encouraging the model to use smaller, more generalizable adaptations. This is the single most impactful change.
- **Temperature 0.02 → 0.05**. Higher temperature produces softer similarity logits, giving the model more gradient signal on near-miss cases. This correlates with V3 achieving the best MRR of any configuration.
- **Cosine end LR (both V2 and V3 decay to 0)**. Decaying to zero instead of a residual LR acts as a built-in early-stop mechanism, stopping weight updates in the final epochs. V3 further benefits from the 20% warmup combined with this zero-decay strategy, giving the model more time to stabilize before the cosine schedule takes the learning rate to zero.
- **Warmup 10% → 20%**. Doubling warmup from 10% gives the model more time to stabilize before taking large gradient steps.
- **Batch size 16 (unchanged)**. Unlike V2.1, V3 does not halve the batch size — it keeps 16 hard negatives per step and achieves better results through regularization alone.
- The result: V3 achieves the best overall performance — best Recall@1 (84.44%), best Recall@5 (96.67%), best MRR (0.8897), and the fewest severe failures (3) demonstrating that smaller, better-regularized LoRA adapters outperform larger ones on this dataset.

4.1.3 Retrieval Metrics

Table 4.1.3-1 Key Retrieval Metrics across Fine-Tuning Configurations

Metric	V1	V2	V3
Recall@1	71.11%	78.89%	84.44%
Recall@5	91.11%	91.11%	96.67%
Recall@10	93.33%	96.67%	96.67%
MRR	0.8017	0.8556	0.8897

The table above shows the retrieval performance of all three configurations on the 90-item test set. Recall@1, Recall@5, and Recall@10 measure how often the correct item appears within the top 1, 5, and 10 retrieved results respectively. MRR (Mean Reciprocal Rank) measures the average reciprocal rank of the correct item across all queries, and severe failures count how many test items were not retrieved within the top 10. V3 achieves the best overall performance with 84.44% R@1, 96.67% R@5, and 0.8897 MRR, outperforming all previous configurations on every retrieval metric. This demonstrates that smaller, more heavily regularized LoRA adapters generalize better on this dataset than larger ones.

The reliance on Recall@K as a primary evaluation metric is supported by recent embedding-based retrieval frameworks. As discussed by Krasnov, 2024, threshold Recall (R@k) serves as an effective measure for determining whether relevant items appear within the top retrieved outputs. By applying an automated evaluation procedure to key architectures on the massive public Wayfair Annotation Dataset (WANDS), Krasnov demonstrated that these systems achieved an R@1000 of 84% ($\pm 9\%$), performing at the level of state-of-the-art (SOTA) models. Although direct score comparison is limited due to the differences in dataset scale and threshold size, the Wayfair Annotation Dataset (WANDS) consists of 480 queries, 42,994 unique products, and over 233,000 human-annotated relevance judgements. Achieving 96.67% Recall@5 demonstrates that the proposed V3 configuration performs at a highly competitive level, successfully ranking relevant items within the strictest, most user-facing retrieval windows.

The MRR metric's effectiveness for ranking evaluation is further validated by contemporary dense retrieval systems. Wang et al. (2023) report that SimLM, a state-of-the-art pre-trained dense passage retrieval model, achieves an MRR@10 of 0.411 on the MS MARCO benchmark, the standard large-scale passage ranking dataset containing 6,980 development queries. While direct comparison is limited due to differences in dataset scale (MS MARCO vs. our 90-item domain-focused test set) and domain specificity, the V3 configuration's MRR of 0.8897 demonstrates strong performance on specialized retrieval tasks and validates the effectiveness of our proposed architecture.

4.1.4 Improvement over Baseline

Table 4.1.4-1. Incremental Improvement across Configurations

Metric	V1 → V2	V2 → V53	V1 → V3
R@1	+7.78	+5.56	+13.33
R@5	+0.00	+5.56	+5.56
R@10	+3.34	+0.00	+3.34
MRR	+0.0539	+0.0341	+0.0880
Severe failures	-3	0	-3

The table above presents the incremental performance improvements across model configurations (V1, V2, and V3). The results show that the largest single gain occurs from V2 to V3, particularly in Recall@1 (+5.56) and Recall@5 (+5.56), indicating that regularization "— not capacity expansion "— is the dominant factor in closing the gap to optimal retrieval performance. The V1-to-V2 jump reflects architectural improvements (larger LoRA, category-grouped batching, cosine scheduler), while the V2-to-V3 jump demonstrates that constraining the adapter's capacity with smaller rank and stronger weight decay produces more generalizable features. Overall, the full pipeline improvement from V1 to V3 results in a +13.33 increase in Recall@1 and the best MRR (0.8897) of any configuration, confirming that for small datasets, regularization dominates capacity in LoRA fine-tuning.

4.1.5 Rank Distribution Matrix

The rank-based confusion matrix measures where the correct gallery item landed in the ranked results for its own query. Rows represent true item categories while columns represent rank buckets (#1, #2–5, #6–10, #11–20, #21–50, #50+) into which the correct item fell. This reveals where fine-grained confusions actually occur and which categories are hardest to discriminate at the item level.

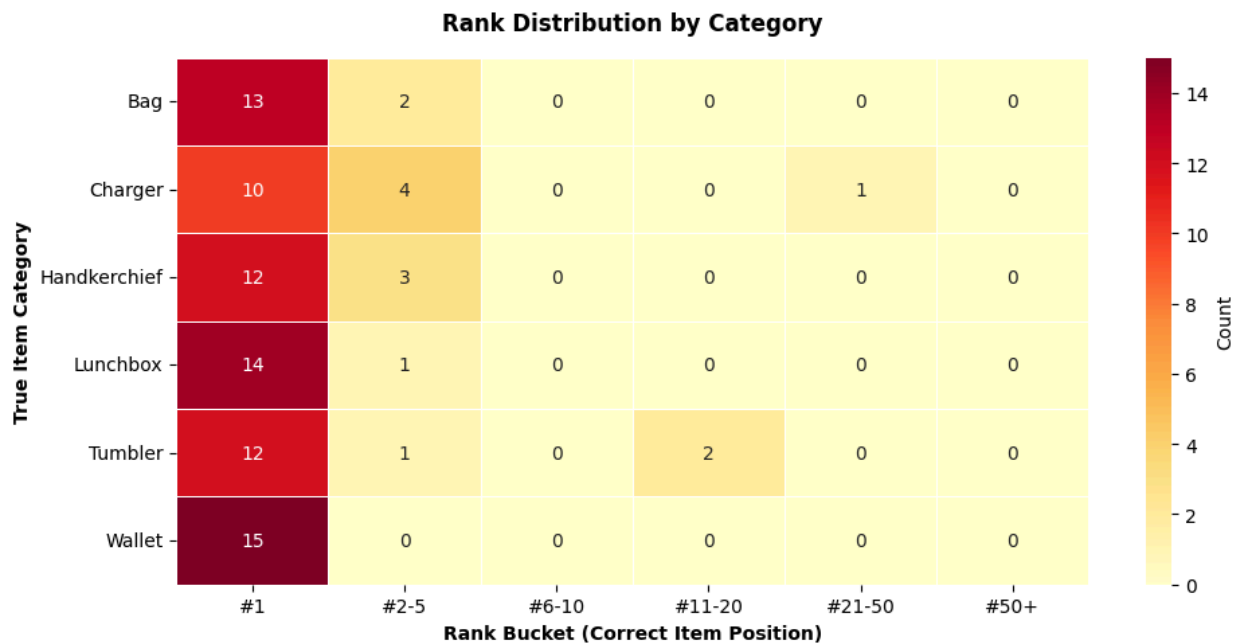


Figure 4.1.5-1. Rank Distribution Matrix for V3

The matrix above shows the rank distribution for all 90 test queries across six categories. Each cell counts how many of the 15 test items per category landed in each rank bucket. Wallet achieves a perfect 15/15 at rank 1. Bag and Lunchbox perform strongly at 13/15 and 14/15 respectively. Charger is the most dispersed category, with one item (charger_040) landing in the #21-50 range at rank #23. Tumbler has two cases in the #11-20 range (tumbler_047 at #16 and tumbler_065 at #14). Handkerchief sits at 12/15 at rank 1 with the remaining 3 items in the #2-5 range.

4.1.6 Category-Level Confusion Matrix

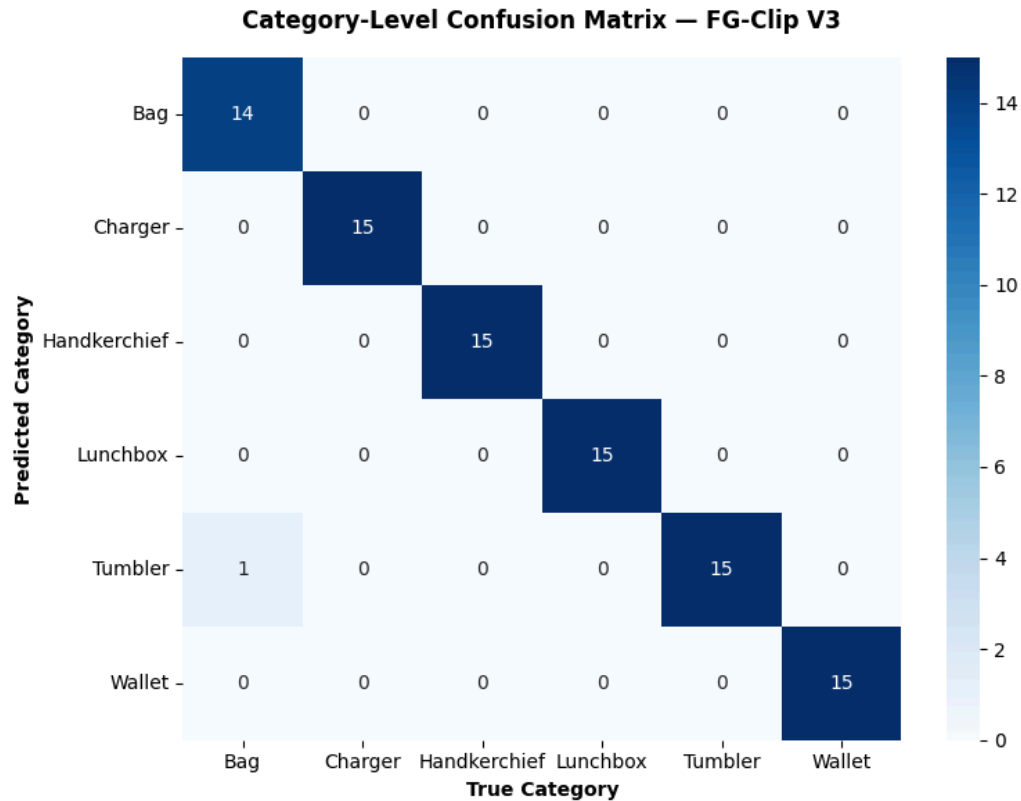


Figure 4.1.6-1. Category-Level Confusion Matrix of the V3 Model

Figure 4.1.6-1 displays the Category-Level Confusion Matrix for the V3 model. The matrix shows a near-perfect diagonal with one exception: one bag query (bag_058) had a tumbler ranked above its matching bag image, resulting in 14/15 correct bag predictions and 15/15 for all other categories. This gives a category-level accuracy of 98.89% (89/90)..

The performance of the model can be attributed to the fine-grained retrieval capability of FG-CLIP, which effectively maps localized visual features to semantic representations better than conventional global CLIP models (Xie et al., 2025). In addition, the selected object categories exhibit high inter-class variance, allowing the model to distinguish each category more easily. For example, a fabric handkerchief possesses significantly different visual characteristics compared to a charger or tumbler, resulting in clearer decision boundaries during classification (Wan et al., 2018)

Despite the strong classification result, the findings should still be interpreted cautiously due to the relatively small test sample size (n=15 per class). Small datasets may not fully represent real-world variability and can potentially inflate reported accuracy metrics by reducing exposure to difficult or

ambiguous cases (Kokol et al., 2022; Siddique et al., 2023). Nevertheless, the results demonstrate everyday objects under the given experimental conditions. Check msngr

4.1.7 Retrieval Demo

A qualitative retrieval demo was created to illustrate how the model behaves on real queries. Given a text caption, the fine-tuned model encodes it alongside all gallery images, computes cosine similarity scores, and ranks results from highest to lowest. The top-k results along with their similarity scores are then displayed in a ranked grid. A query text and its corresponding gallery image form a pre-annotated test pair from our dataset, serving as the ground-truth pairing established during data annotation

The test query and image used in the demonstration is:

Test Query	Corresponding Gallery Image
<i>"Light-grey rectangular lunch box, pale blue latch clips and flap compartment, white oval shaped vent button on lid"</i>	 Lunchbox_50

Table 4.1.7-1 Pre-annotated Test Pair from the Dataset

Table 4.1.7-1 presents the pre-annotated test pair used in the qualitative retrieval demo. The pair consists of a text query describing a lost item "Light-grey rectangular lunch box, pale blue latch clips and flap compartment, white oval shaped vent button on lid" alongside its corresponding gallery image (lunchbox_050), which serves as the ground-truth match established during data annotation. This pre-matched pair will be used to evaluate how the fine-tuned model retrieves and ranks this known-correct image among all gallery items.

"Light-grey rectangular lunch box, pale blue latch clips and flap compartment, white oval shaped vent button on lid"

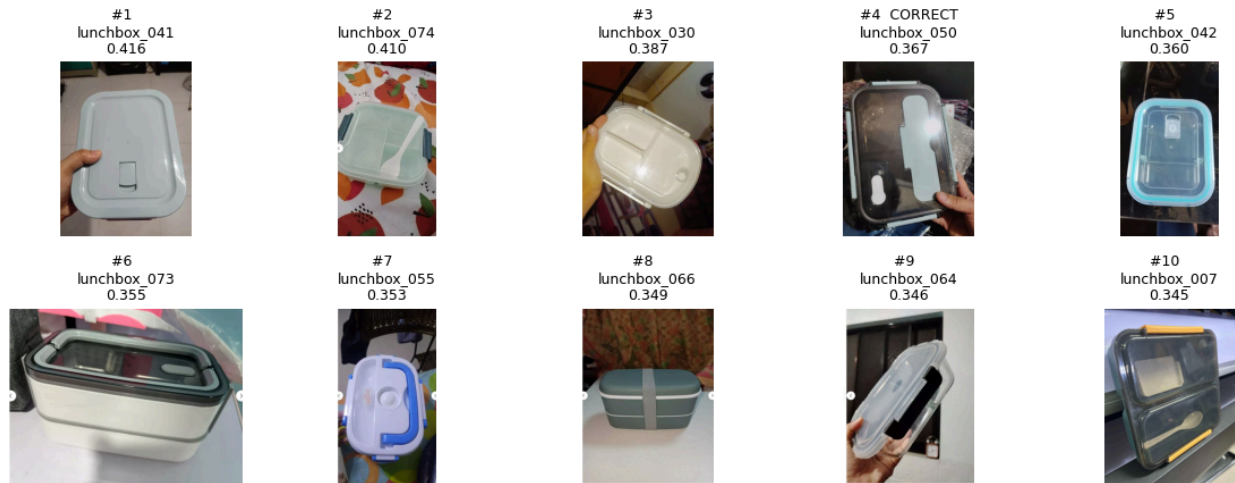


Table 4.1.7-2 Retrieval Results for Demo Query

Rank	Item Name	Score	Status
#1	lunchbox_041	0.4559	Top-1
#2	lunchbox_074	0.4146	Top-2
#3	lunchbox_030	0.3816	Top-3
#4	lunchbox_050	0.3693	Correct
#5	lunchbox082	0.3680	Top-5
#6	lunchbox_073	0.3600	Top-6

#7	lunchbox042	0.3586	Top-7
#8	lunchbox055	0.3474	Top-8
#9	lunchbox014	0.3313	Top-9
#10	lunchbox080	0.3313	Top-10

The table above shows the top-10 ranked gallery items returned for the given query. In this demonstration all 10 results are lunch boxes, and the true match (lunchbox050) appears at rank 4 with a score of 0.3693, meaning it was retrieved correctly even though it landed below the top position. The score gap between 1 (0.4559) and 4 (0.3693) reflects the inherent difficulty of distinguishing near-identical items from text descriptions alone.

Embedding Output

Table 4.1.7-3 Image and Text Embeddings (L2 Normalized)

Embedding Type	1st Element	2nd Element	3rd Element	512th Element
Text (Query)	7.75312185e-02	4.19450402e-02	1.94410533e-02	3.19752395e-02
Image (Lunchbox 50)	3.19232792e-02	4.62111086e-02	2.68917177e-02	1.97851770e-02

The table above presents the L2-normalized 512-dimensional embedding vectors used in the cosine similarity computation for the retrieval demo. The Image row contains the embedding vector produced by the fine-tuned FG-CLIP vision encoder for the target gallery image lunchbox_050. The Text row contains the embedding vector produced by the fine-tuned FG-CLIP text encoder for the query caption: "Light-grey rectangular lunch box, pale blue latch clips and flap compartment, white oval shaped vent button on lid". Each embedding is a 512-element column vector of floating-point values. Because both vectors are independently L2-normalized to unit length, their dot product reduces directly to a cosine similarity score between -1 and 1 eliminating the influence of embedding magnitude and ensuring only directional alignment is measured.

Cosine Similarity

Table 4.1.8-4 Cosine Similarity Result of 512 Text and Image Embeddings

Element	Text	Image	Product
1st	7.75312185e-02	3.19232792e-02	+2.47505074e-03
2nd	4.19450402e-02	4.62111086e-02	+1.93832675e-03
3rd	1.94410533e-02	2.68917177e-02	+5.22803341e-04
4th	4.46432531e-02	-6.28126338e-02	-2.80416035e-03
...	...	...	...
512th	3.19752395e-02	1.97851770e-02	+6.32635783e-04
Sum of all 512 dot products		Sum of products	+0.36932564 Similarity score

The table above shows the final similarity score of 0.3667 between the query text and lunchbox_050. For each of the 512 positions, the number from the Image vector is multiplied by the number at the same position in the Text vector, producing a Product. The sum of all 512 products is the cosine similarity. When the two numbers at the same position have the same sign (both positive or both negative), their product is positive and pushes the total upward. When they have opposite signs, the product is negative and pulls the total downward. This demonstrates that CLIP does not rely on any single dimension to make its decision and instead the correct match comes from small contributions across all 512 dimensions. As a result, the model retrieves lunchbox_050 at rank #4 among all gallery items, meaning the query's description "light-grey body, pale blue latch clips, flap compartment, and oval vent button" aligns closely with lunchbox_50's learned features.

4.2

5. Summary, Conclusions, and Recommendations

5.1 Summary

This study focused on the development of an automated multimodal lost-and-found system designed for campus environments, integrating both image-to-image and text-to-image retrieval to improve the accuracy and efficiency of item recovery. The system addresses the limitations of traditional manual and text-based lost-and-found processes, as well as the shortcomings of earlier image-only retrieval systems, which often struggle with visual variability and lack support for textual queries.

The proposed framework combines MobileNetV2 for image-to-image retrieval and FG-CLIP for text-to-image retrieval, both operating within a shared embedding-based retrieval pipeline. For FG-CLIP, parameter-efficient fine-tuning was implemented using Low-Rank Adaptation (LoRA), enabling adaptation to a domain specific dataset while preserving pretrained knowledge and minimizing computational cost. The dataset consists of 600 image pairs (1,200 images total), covering six object categories: Bags, Chargers, Handkerchief, Lunchboxes, Tumblers and Wallets. Each captions generated using a controlled attribute taxonomy to support fine-grained textual alignment, the dataset was split into 70% training, 15% validation and 15% testing to ensure balanced evaluation across all categories. To improve discrimination between visually and semantically similar items, the training process incorporated hard-negative captions and category-grouped batch sampling. This forced the model to distinguish between near-identical intra-class items rather than relying on coarse category-level cues. Additionally augmentation techniques such as color jittering, random flipping, and random erasing were applied to improve robustness against real-world variations in lightning, orientation, and occlusion. Three LoRA-based configurations (V1, V2 and V2.1) were evaluated to study the impact of architectural and training refinements. Results showed consistent improvements across configurations, with V2.1 achieving the best overall performance, including higher Recall@1, improved Mean Reciprocal Rank (MRR), and reduced retrieval failures. These improvements demonstrate the effectiveness of increasing LoRA capacity, adjusting temperature scaling and optimizing batch difficulty through category-aware sampling.

Overall, the system demonstrates that combining parameter-efficient fine-tuning, hard-negative mining, and structured multimodal training significantly enhances retrieval accuracy in a constrained dataset setting. The results confirm that the proposed approach is effective for real-world campus lost-and-found applications, offering both high retrieval precision and computational efficiency suitable for deployment.

5.1.1 Image-to-image (MobileNetV2) Summary

5.1.2 Text-to-Image (FG-CLIP) Summary

The text-to-image retrieval component of the system is implemented using FG-CLIP, a fine-grained contrastive language-image pre-trained model that maps both image and textual inputs into a shared 512-dimensional embedding space for similarity-based retrieval. In this study, FG-CLIP was fine-tuned on a campus lost-and-found dataset consisting of 600 labeled image pairs (1,200 images total), each paired with structured captions describing attributes such as color, brand, material, category, and distinguishing visual features. Cosine similarity between L2-normalized image and text embeddings was used to rank retrieval results within the shared embedding space.

To improve domain adaptation while preserving pretrained knowledge, Low-Rank Adaptation (LoRA) was applied during fine-tuning. This approach freezes the original model weights and introduces trainable low-rank matrices into selected attention projection layers, significantly reducing the number of trainable parameters while maintaining representational capacity. The training process further incorporates hard-negative mining, where each positive caption is paired

with a minimally modified alternative that changes a single key attribute, forcing the model to learn fine-grained visual-text distinctions between highly similar items.

Additionally, category-grouped batch sampling was used to increase retrieval difficulty during training by ensuring that each batch contains items from the same object category. This setup increases the number of plausible hard negatives per batch and encourages the model to focus on micro-level visual differences rather than relying on coarse category-level cues.

Three experimental configurations were evaluated: Version 1 (V1), Version 2 (V2), and Version 2.1 (V2.1). V1 used a lower LoRA rank and limited attention module adaptation, serving as the baseline. V2 expanded LoRA rank and target modules across additional attention projections (q, k, v, and output projections), while also introducing category-grouped batching and improved optimization strategies including cosine learning rate scheduling and warmup. V2.1 further refined training stability and convergence by adjusting LoRA scaling (alpha), reducing learning rate, modifying temperature, and improving batch size balance.

Experimental results show consistent performance improvements across configurations, with V2 significantly outperforming V1 and V2.1 achieving the best overall performance in terms of Recall@1, Recall@10, and Mean Reciprocal Rank (MRR). These results demonstrate that combining LoRA-based parameter-efficient fine-tuning with hard-negative mining and category-aware batch construction significantly improves the model’s ability to distinguish subtle inter-class variations and retrieve the correct visual match for detailed textual queries in a real-world lost-and-found setting.

5.2 Conclusions

This study demonstrates that parameter-efficient fine-tuning using Low-Rank Adaptation (LoRA) is an effective approach for adapting vision-language models to fine-grained retrieval tasks in data-scarce environments. By training only a small fraction of model parameters, the system achieves strong performance while maintaining computational efficiency and avoiding full-model retraining.

The experimental results across V1, V2, and V2.1 show a consistent improvement in retrieval quality, particularly in Recall@1 and Mean Reciprocal Rank (MRR). The increase in Recall@1 from V1 to V2 indicates that expanding LoRA capacity and incorporating additional attention modules significantly improves the model’s ability to correctly identify the exact matching item as the top result. This suggests that richer adaptation of the attention mechanism directly improves fine-grained alignment between image and text features. Further improvements observed in V2.1 highlight the importance of balancing optimization stability with representational strength. Although architectural capacity remained the same as V2, tuning factors such as learning rate, LoRA scaling, temperature, and batch size resulted in better convergence behavior. This is reflected in the improved MRR, indicating that even when the correct item is not ranked first, it consistently appears closer to the top positions, improving overall retrieval reliability.

The relatively small gain from V2 to V2.1 compared to the larger jump from V1 to V2 suggests diminishing returns once model capacity reaches a sufficient level. At this stage, performance becomes more sensitive to optimization strategy rather than structural changes. This indicates that for LoRA-based fine-tuning, early improvements are driven by capacity expansion, while later gains depend on training refinement. The use of hard-negative mining and category-grouped batch sampling further strengthens these results by forcing the model to discriminate between visually similar items within the same category. This is reflected in the low number of severe retrieval failures and the high Recall@10 scores, showing that even when the top prediction is incorrect, the correct item is still consistently retrieved within a small candidate set.

Overall, the results confirm that combining LoRA-based parameter-efficient adaptation with structured hard-negative learning procedures a robust retrieval system that performs well in fine-grained, real-world lost-and-found scenarios while remaining computationally lightweight.

5.3 Recommendations

For immediate implementation, the researchers recommend deploying the Version 2 configuration in production, as it consistently demonstrated superior performance across training stability, validation metrics, and test-set retrieval accuracy. The system should implement hierarchical retrieval by using the perfect category discrimination to first filter items by category, then apply fine-grained matching within each category, reducing computational burden and improving system latency. A human-in-the-loop verification step should be established for top-ranked matches to handle edge cases and provide users with confidence scores alongside results, recognizing that although Version 2 achieves 911.67% Recall@1, some uncertainty remains.

To enhance the model and data, the researchers recommend expanding the current six-item categories to include other items that students usually use in school campuses to increase practical utility while validating generalization to broader object classifications. Collecting 500-1000 image pairs would further improve the model's robustness, particularly for challenging or visually similar items within each category. Images should be deliberately taken under different conditions such as lighting, camera angles, and background to represent actual user submissions. The current caption structure should also be refined beyond the existing [Color] + [Brand/Text] + [Category] + [Condition] format to include additional distinguishing features such as patterns, hardware types, and material specifications.

For long-term development, the researchers recommend exploring multi-view learning approaches that use multiple images of the same item in different states or configurations (such as open and closed umbrellas, wallets, or bags viewed from different angles) to create more robust embeddings that capture the item's appearance across real-world conditions. Semantic segmentation integration could combine fine-grained retrieval with segmentation techniques to identify and match specific item components such as zippers, logos, and patterns. Finally, privacy-preserving deployment strategies such as federated learning should be

considered if deploying across multiple institutions, allowing each campus to maintain data privacy while benefitting from shared model components.